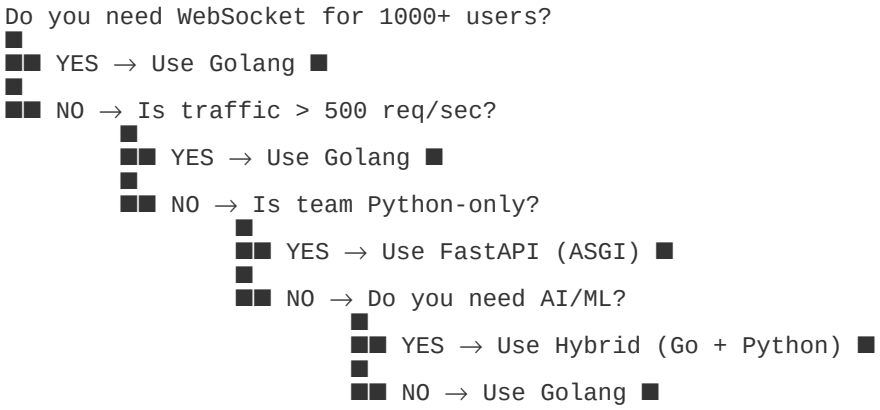
Architecture Decision Matrix

Quick reference for choosing between ASGI (Python) and Golang backends.

TL;DR

Go wins for: High traffic, WebSocket, real-time, job queues, cost
Python wins for: Quick prototypes, Python-only teams, low traffic
Hybrid wins for: Production apps with AI (our choice)

Decision Tree



Comparison Matrix

Feature ASGI (FastAPI) Golang Winner Performance Requests/sec 2,000-5,000 20,000-50,000 Go 10x WebSocket connections 500-1,000 10,000+ Go 10x Latency (p99) 200-500ms 20-50ms Go 10x Memory (1K users) 800 MB 50 MB Go 16x Development Learning curve Medium Medium Tie Code complexity High (async) Low Go Debugging Hard (async) Easy Go Testing Medium Easy Go Ecosystem Web frameworks FastAPI ■ Gin, Echo Tie AI/ML libraries Excellent ■ Poor Python Async libraries Many Built-in Go Package

management pip/poetry go mod Go Deployment Binary size N/A 10-20 MB Go Docker image 200-300 MB 10-20 MB Go 15x Startup time 2-5 sec instant Go Cross-compile No Yes ■ Go Dependencies Complex Simple Go Operations Monitoring Good Excellent Go Profiling Medium Built-in ■ Go Resource usage High Low Go Scaling Horizontal Vertical+Hor Go Cost Server (1K users) $100-150/mo $10-20/mo Go 10x Server (10K users) $1,500/mo $50-100/mo Go 15x Maintenance Higher Lower Go Use Case Matrix

■ *Use ASGI (FastAPI)*

Use Case Why Internal dashboard Low traffic, quick to build ML inference API Python ecosystem, no workers needed CRUD API (< 100 users) Simple, good enough Prototype/MVP Fast development (1-2 days) Data processing API Python data libs (pandas, numpy) Admin tools Python-only team, low stakes **Example:**

```
# Quick prototype in 30 minutes
from fastapi import FastAPI
app = FastAPI()

@app.post("/predict")
async def predict(data: dict):
    result = ml_model.predict(data)
    return {"result": result}

# Perfect for: Internal ML dashboard, 10-50 users
```

■ *Use Golang*

Use Case Why Real-time collaboration WebSocket, low latency Chat application 10K+ connections API Gateway High throughput, routing Job queue orchestrator Concurrent job management File upload/download Streaming, parallel processing Microservices Small footprint, fast startup Live dashboard WebSocket broadcast to 1000s IoT backend Low memory, high connections **Example:**

```
// Production-ready in 2-3 days
func main() {
    r := gin.Default()

    r.GET("/ws", WebSocketHandler)
    r.POST("/api/upload", UploadHandler)
    r.GET("/api/stream", StreamHandler)

    r.Run(":8080")
}

// Perfect for: Real-time apps, high traffic
```

■ *Use Hybrid (Go + Python)*

Use Case Why AI content generation Go for API, Python for AI Image processing service Go for upload, Python for processing Real-time ML inference Go for WebSocket, Python for ML Data enrichment pipeline Go for orchestration, Python for ETL Recommendation engine Go for API, Python for algorithms Video processing Go for streaming, Python for encoding **Example:**

```
Go Backend (Port 8080)      Python Workers
  ■■ POST /generate    →    Redis Queue → Groq API
  ■■ WebSocket /ws     →    Real-time updates
  ■■ GET /status       →    Redis cache

Perfect for: Production AI apps
```

Traffic-Based Recommendation

< 100 Concurrent Users

Recommendation: FastAPI (ASGI)
- Cost: $10-20/month
- Deployment: Single Uvicorn instance
- Why: Simple, good enough, fast to build

100-1000 Concurrent Users

Recommendation: Golang
- Cost: $15-30/month
- Deployment: Single Go instance
- Why: Better performance, still cheap

1000-10,000 Concurrent Users

Recommendation: Golang + Load Balancer
- Cost: $50-100/month
- Deployment: 2-3 Go instances
- Why: Proven at scale, cost-effective

10,000+ Concurrent Users

Recommendation: Golang + Kubernetes
- Cost: $200-500/month
- Deployment: Auto-scaling cluster
- Why: Enterprise-grade, highly available

Feature-Based Recommendation

WebSocket Usage

Connections Recommendation Why < 100 FastAPI Good enough 100-1000 Golang Better performance 1000+ Golang Only viable option Job Queue

Jobs/sec Recommendation Why < 10 FastAPI Simple 10-100 Golang Better orchestration 100+ Golang Parallel processing File Processing

File Size Recommendation Why < 10 MB FastAPI Works fine 10-100 MB Golang Streaming 100+ MB Golang Memory efficient Cost Breakdown (AWS EC2)

ASGI Stack (FastAPI)

1,000 users:
- Instance: t3.large (2 CPU, 8 GB)
- 4 Uvicorn workers
- Cost: **$60/month**

10,000 users:
- 8x t3.large instances
- Load balancer
- Cost: **$600/month**

Golang Stack

1,000 users:
- Instance: t3.small (1 CPU, 2 GB)
- Single Go process
- Cost: **$15/month**

10,000 users:
- 2x t3.medium (2 CPU, 4 GB)
- Load balancer

- Cost: **$80/month**

Savings: 85-90%

Development Speed

FastAPI (Quick prototype)

```
Day 1: API with 5 endpoints ■
Day 2: Deploy to production ■
Total: 2 days
```

Golang (Production-ready)

```
Day 1-2: API with 10 endpoints ■
Day 3: WebSocket + job queue ■
Day 4: Testing + deployment ■
Total: 4 days
```

Verdict: FastAPI 2x faster for MVP, but Go scales better

Team Skills Matrix

Python-only team

Recommendation: FastAPI → Then learn Go for v2
- Start with FastAPI for speed
- Migrate to Go when scaling issues appear
- Incremental learning curve

Mixed skills team

Recommendation: Hybrid (Go + Python)
- Go developers: Backend
- Python developers: AI workers
- Best of both worlds

Go-experienced team

Recommendation: All Golang
- Except AI/ML (use Python workers)
- Consistent codebase
- Simpler deployment

Real-World Scenarios

Scenario 1: Startup MVP

Requirements:
- Build in 1 week
- 10-50 early users
- Tight budget

Recommendation: FastAPI ■
Why: Speed to market matters more than performance

Scenario 2: Growing SaaS

Requirements:
- 500 active users (growing)
- Real-time features needed
- Need to control costs

Recommendation: Golang ■
Why: Will scale smoothly, cheaper long-term

Scenario 3: Enterprise Platform

Requirements:
- 10,000+ concurrent users
- SLA requirements
- AI-powered features

Recommendation: Hybrid (Go + Python) ■
Why: Best performance + best AI ecosystem

Scenario 4: AI Content Generator (Our Project!)

Requirements:
- 1000+ concurrent WebSocket users
- Text/image generation (AI)
- Real-time progress updates
- Batch processing

Recommendation: Hybrid (Go + Python) ■

Why:
1. ■ Golang handles WebSocket (10K connections)
2. ■ Golang orchestrates job queue
3. ■ Python handles AI (Groq, Stable Diffusion)
4. ■ 90% cost savings vs all-Python
5. ■ Better user experience (low latency)

Migration Strategy

Start Small → Scale Up

Phase 1: MVP (FastAPI)

```
# Week 1: Launch quickly
@app.post("/generate")
async def generate(prompt: str):
    result = groq_api.call(prompt)
    return result

# Good for: Validating idea
# Supports: 10-100 users
```

Phase 2: Add Queue (Still FastAPI)

```
# Month 2: Handle more traffic
@app.post("/generate")
async def generate(prompt: str):
    job_id = await redis.lpush("queue", prompt)
    return {"job_id": job_id}

# Worker.py processes queue
# Supports: 100-500 users
```

Phase 3: Go Backend (Hybrid)

```
// Month 6: Real scale
func Generate(c *gin.Context) {
    jobID := createJob(prompt)
    c.JSON(202, gin.H{"job_id": jobID})
}

// Python workers stay same!
// Supports: 1000+ users
```

Common Mistakes

■ *Using FastAPI for WebSocket at scale*

Problem: Event loop saturates at 500-1000 connections
Solution: Use Golang from the start

■ *Using Golang for ML inference*

Problem: Poor ML ecosystem, no good libraries
Solution: Use Python workers for AI

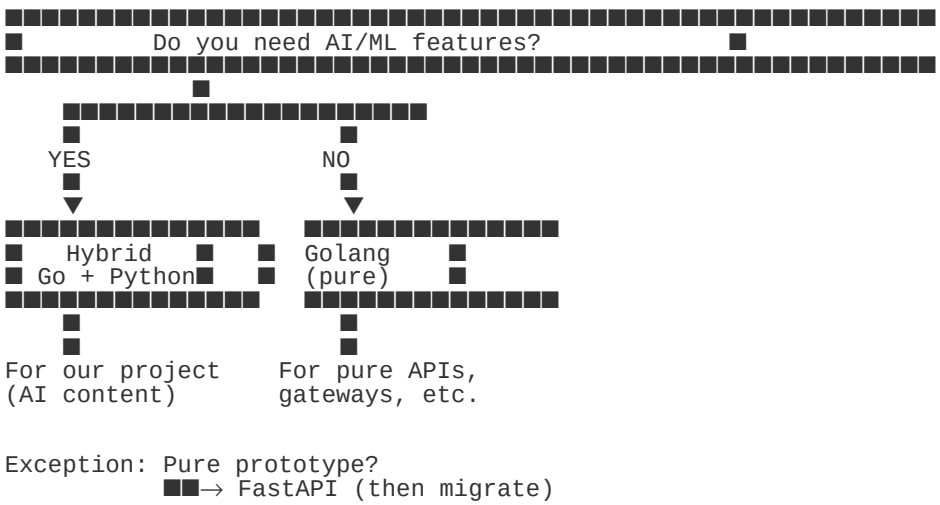
■ *Over-engineering MVP*

Problem: Building Go backend when 10 users
Solution: Start with FastAPI, migrate later

■ *Under-engineering for production*

Problem: Using FastAPI for 1000+ users
Solution: Migrate to Go before issues appear

Final Decision Chart



Recommendation for Creative Automation Hub

■ *Hybrid Architecture (Go + Python)*

Why this wins:

Performance:
- 10x faster API responses (Go)
- 10x more WebSocket connections (Go)
- Real-time updates with < 50ms latency (Go)

AI Capabilities:
- Best ML ecosystem (Python)
- Easy Groq/Anthropic integration (Python)
- Stable Diffusion support (Python)

Cost:
- 90% cheaper than all-Python
- $15/month vs $150/month for 1000 users

User Experience:
- Smooth real-time updates
- No lag during job processing
- Professional feel

Developer Experience:
- Simple Go code (no async complexity)
- Familiar Python for AI
- Clean separation of concerns

Score: 10/10 ■

Quick Reference

Metric FastAPI Golang Hybrid MVP Speed ■■■■■ ■■■ ■■■■ Performance ■■ ■■■■■ ■■■■■ WebSocket ■■ ■■■■■ ■■■■■ AI/ML ■■■■■ ■ ■■■■■ Cost ■■ ■■■■■ ■■■■■ Scalability ■■ ■■■■■ ■■■■■ Simplicity ■■■■ ■■■■ ■■■ **Overall Winner: Hybrid ■**